

NAME:PRAJWAL G T

SRN:PES2UG25CS818

GAME NAME: 61_math_flashcards

Task 1: Fix the string concatenation calculation bug

Entering the mathematically correct answer is judged as incorrect by the game. In `game_engine.compute_expected_answer()`, the function returns `int(f"{self.num_a}{self.num_b}")` instead of evaluating the actual arithmetic operation. For example, `7 + 5` expects 75 rather than 12. Refactor `compute_expected_answer()` to check `self.operator` and compute the correct mathematical result using `+`, `-`, or `*`.

BEFORE UPDATE THE CODE:

```
def compute_expected_answer(self):

    # BUG SYMPTOM:
    # Operands are combined as strings instead of evaluated with mathematical operations.
    return int(f"{self.num_a}{self.num_b}")
```

AFTER UPDATE THE CODE:

```
def compute_expected_answer(self):
    if self.operator == "+":
        return self.num_a + self.num_b
    elif self.operator == "-":
        return self.num_a - self.num_b
    elif self.operator == "*":
        return self.num_a * self.num_b
    elif self.operator == "/":
        return self.num_a // self.num_b
    raise ValueError(f"Unsupported operator: {self.operator}")
```

Task 2: Implement a per-question timer bar

Flashcards are more engaging when speed is tested. Add a visible countdown timer bar (e.g., 10 seconds) below the card container in `game_engine.render()`. Update the timer in `game_engine.update()`, and if the time runs out before the player submits, automatically count the card as an incorrect attempt, display a "TIME'S UP!" warning, and move to the next card.

BEFORE UPDATE THE CODE:

```
def update(self):
    # Original implementation before Task 2: pass
    current_time = pygame.time.get_ticks()
    delta_time = (current_time - self.last_update_time) / 1000
    self.last_update_time = current_time
    self.time_remaining = max(0, self.time_remaining - delta_time)
```

AFTER UPDATE THE CODE:

```
def update(self):
    # Original implementation before Task 2: pass
    current_time = pygame.time.get_ticks()
    delta_time = (current_time - self.last_update_time) / 1000
    self.last_update_time = current_time
    self.time_remaining = max(0, self.time_remaining - delta_time)

    if self.time_remaining == 0:
        self.total_attempts += 1
        self.streak = 0
        self.feedback_msg = "TIME'S UP!"
        self.feedback_color = (240, 75, 75)
        self.generate_new_card()
        return True
    return False
```

Task 3: Implement consecutive correct streak multipliers

Currently, every correct answer simply awards a flat +1 point. Implement a consecutive streak tracker in `game_engine`. Maintain a multiplier that increases for every consecutive correct response (e.g., 2x score for 3 in a row, 3x score for 5 in a row), and reset the streak counter to zero on any incorrect answer or timeout.

BEFORE UPDATE THE CODE:

```
if user_answer == expected:
    # Original scoring before Task 3: self.score += 1
    self.streak += 1
    if self.streak >= 5:
        multiplier = 3
    elif self.streak >= 3:
        multiplier = 2
    else:
        multiplier = 1
```

AFTER UPDATE THE CODE:

```
if user_answer == expected:
    # Original scoring before Task 3: self.score += 1
    self.streak += 1
    if self.streak >= 5:
        multiplier = 3
    elif self.streak >= 3:
        multiplier = 2
    else:
        multiplier = 1
    self.score += multiplier
    self.feedback_msg = f"CORRECT! {self.num_a} {self.operator} {self.num_b} = {expected}"
    self.feedback_color = (80, 230, 110)
    self.generate_new_card()
else:
    self.streak = 0
    self.feedback_msg = f"WRONG! Expected {expected}."
    self.feedback_color = (240, 75, 75)
    self.input_box.clear()
```

Task 4: Implement division operator support with integer results

Flashcards only test addition, subtraction, and multiplication. Add the integer division operator (/) into `game_engine.generate_new_card()`. Ensure that cards generated with division always produce clean whole numbers without remainders (for example, by generating a divisor and quotient first, then multiplying them to produce the dividend).

BEFORE UPDATE THE CODE:

```
def generate_new_card(self):
    # Original card generation before Task 4:
    # self.num_a = random.randint(3, 15)
    # self.num_b = random.randint(2, 12)
    # self.operator = random.choice(["+", "-", "*"])
```

AFTER UPDATE THE CODE:

```
def generate_new_card(self):
    # Original card generation before Task 4:
    # self.num_a = random.randint(3, 15)
    # self.num_b = random.randint(2, 12)
    # self.operator = random.choice(["+", "-", "*"])
    self.operator = random.choice(["+", "-", "*", "/"])
    if self.operator == "/":
        self.num_b = random.randint(2, 7)
        quotient = random.randint(2, 15 // self.num_b)
        self.num_a = self.num_b * quotient
    else:
        self.num_a = random.randint(3, 15)
        self.num_b = random.randint(2, 12)
    if self.operator == "-" and self.num_a < self.num_b:
        self.num_a, self.num_b = self.num_b, self.num_a
```